

// JavaJub

Собес в Бизнес Технологиях

Global ERP • альтернатива SAP

Java • Junior

Часть 1 • бесплатная версия • @Java_Jub

Вопросы, задачи и подготовка

к интервью: Java Core + Алгоритмы + Обход деревьев

Стек по вакансии

Java SE • ООП • Коллекции • HTTP • PostgreSQL

`equals/hashCode` • `String Pool` • `Collections` • Алгоритмы • Деревья • REST

Содержание

Разделы 01–09 — открыты. 10–22 — в *Java Jub Pro* (показаны темы и вопросы).

01	Бизнес Технологии и Global ERP	3
02	Формат собеседования	5
03	Стек по вакансии: что готовить	7
04	Java Core	9
05	Коллекции	15
06	ООП, SOLID и принципы	18
07	Исключения	21
08	HTTP и REST	23
09	Git, Maven и инструменты	26
10	JVM, память и сборщик мусора	Pro
11	Многопоточность	Pro
12	Spring и Spring Boot	Pro
13	SQL и базы данных	Pro
14	JDBC и работа с БД из Java	Pro
15	Паттерны проектирования	Pro
16	Тестирование: JUnit, Mockito, AssertJ	Pro
17	Алгоритмы и Big-O	Pro
18	Обход деревьев	Pro
19	Live coding: Java-задачи	Pro
20	Code Review с разбором	Pro
21	ERP-домен и базовый web / fullstack	Pro
22	Pet-проекты, soft skills и чек-лист	Pro

01 Бизнес Технологии и Global ERP

Бизнес Технологии — российский разработчик корпоративного ПО с офисом в Санкт-Петербурге на Коломяжском, 25 лет на рынке. Главный продукт — Global ERP: платформа для управления крупным предприятием, позиционируется как реальная отечественная альтернатива SAP и Oracle E-Business Suite. В сценарии импортозамещения это один из ключевых игроков рынка, особенно для машиностроения, судостроения, нефтегаза и химии.

Технологически Global ERP — это трёхзвенная архитектура: PostgreSQL как СУБД, сервер приложений собственной разработки на Java, веб-клиент в браузере. Никакого Spring «из коробки» — у компании свой framework, написанный годами под их задачи. Поэтому на собеседовании тебя проверяют на знание базы Java и алгоритмов, а не на «расскажи Spring Boot 3».

► Что важно понимать про продукт

Что такое Global ERP в одной фразе?

Это российская комплексная информационная система для управления предприятием: финансы, склад, закупки, производство, кадры, документооборот — всё на единой платформе, работающей через браузер. Позиционируется как замена SAP ERP / S4HANA и Oracle EBS. В декабре 2024 платформа выдержала нагрузочный тест на 15 тысяч одновременных пользователей в одной базе — это уровень крупных предприятий.

Какие ещё продукты у компании?

Линейка Global: Global ERP (управление предприятием), Global MES (управление производственными процессами на цеховом уровне — внедрено на НПП «Радар ммс»), Global EAM (управление ремонтами и оборудованием), Global HRM (кадры и расчёт зарплаты), Global-FrameWork (платформа цифровизации управленческих процессов). Все продукты включены в реестр российского ПО.

На каких отраслях специализируется компания?

Машиностроение, судостроение, химическая промышленность, нефтегазовая отрасль, энергетика — то есть тяжёлая индустрия и крупные корпорации с десятками тысяч сотрудников. Это не банковский / e-commerce домен: задачи другие — длинные бизнес-операции, сложная учётная политика, интеграции с ФНС, банками, ГИС, 1С.

Почему у них собственный framework, а не Spring?

Платформа разрабатывалась более 10 лет назад, когда команда сознательно выбрала путь собственных решений, чтобы получить полный контроль над архитектурой и независимость от внешних вендоров. Свой framework позволяет глубоко кастомизировать поведение системы под каждого крупного заказчика — это критично для ERP, где у каждого предприятия свои бизнес-правила. Для тебя это значит: на работе будешь учить их внутренние библиотеки, а не Spring.

Что значит «нет legacy» в описании вакансии?

У компании нет наследия старого Java-кода 2005 года с нечитаемыми JSP-страницами и Struts 1. Платформа писалась с нуля под современный стек, постоянно рефакторится силами команды senior+ уровня. Тебе как Junior это плюс: код будет читабельный, архитектура внятная, ментор в команде есть. Минус: высокая планка к качеству твоего кода тоже.

Чем компания отличается от 1С:ERP и Галактики?

1С и Галактика — это другие российские ERP-системы, исторически сильные в среднем бизнесе и бухгалтерии. Global ERP более ориентирован на крупный enterprise с тяжёлой индустрией и пытается закрыть нишу импортозамещения именно SAP-уровня, где 1С исторически не доминирует. Технологически тоже разные: 1С строится на собственном языке программирования 1С, Global ERP — это Java + PostgreSQL.

Что значит «25 лет на рынке» для меня как Junior?

Компания стабильная, не стартап. Есть процессы, есть ментор, есть документация. Минус — может быть консервативный темп: ревью кода долгое, релизы по графику, эксперименты ограничены. Плюс — белая зарплата, ДМС, аккредитация IT-компании (отсрочка от армии), реальные крупные клиенты, чьи имена идут в резюме весомо.

СОВЕТ • Что почитать перед собесом про продукт

Зайди на global-system.ru — посмотри страницу Global ERP, прочитай 2–3 абзаца про модули и архитектуру. На собесе оценивают, понимаешь ли ты, куда идёшь работать. Полезно знать слова: трёхзвенная архитектура, импортозамещение, реестр российского ПО, машиностроение, судостроение.

► Команда и условия

Сколько человек в компании и где офисы?

Около 400+ сотрудников. Главный офис в Санкт-Петербурге (Коломяжский, 33к2, около метро Пионерская), есть офисы в Перми, Нижнем Новгороде, Вологде. Вакансия Junior — конкретно СПб офис, удалёнка не предлагается. График 5/2 с 9:00 до 18:00, есть гибкость ± 1 час по началу дня.

Что в команде проекта?

Команда состоит из senior и middle+ разработчиков — это значит, что Junior — единственный новичок, остальные опытные. С одной стороны, есть у кого учиться: тебя не оставят один на один с фреймворком. С другой стороны, планка кода высокая, на ревью будут указывать на много мелочей. Это нормально и полезно для роста.

Что важно знать про условия?

Оформление по ТК РФ, полностью белая зарплата от 80 000 рублей на руки для Junior, ДМС после трёх месяцев испытательного срока. Компания аккредитована Минцифры как IT-компания (важно для отсрочки и IT-ипотеки). Есть корпоративные и спортивные мероприятия, pizza-day в последнюю пятницу месяца — это про культуру, и упоминание этого на собесе показывает, что ты читал вакансию.

02 Формат собеседования

Junior-собес в Бизнес Технологиях идёт стандартно для российских продуктовых компаний с офисом: 2 этапа, суммарно 1,5–2 часа, может добавиться тестовое задание. Знание этого формата заранее снимает половину волнения — ты приходишь с пониманием, что именно от тебя ждут на каждом этапе.

► Этапы и длительность

Этап	Длительность	С кем	Содержание
1. HR-скрининг	20–30 мин	Рекрутер	Опыт, мотивация, ожидания, готовность к офису СПб
2. Техническое интервью	1–1,5 часа	Тимлид + СТО / senior разработчик	Java Core + ООП + коллекции + JVM + алгоритмы + SQL + Code Review + pet-проекты + soft
3. Тестовое задание (опц.)	1–3 дня дома	Самостоятельно	Реализация структуры данных, простой REST-сервис, парсер строки или CSV

► Этап 1 — HR-скрининг

Что спрашивает HR на скрининге?

Стандартный набор: расскажи о себе, что заканчивал, какие пет-проекты делал, почему Java, почему именно Бизнес Технологии, готов ли работать в офисе на Пионерской пять дней в неделю. Дополнительно — ожидания по зарплате (вилка от 80 000 на руки, не стесняйся назвать чуть выше нижней границы — например, 90 000), готовность к full-time. Длится 20–30 минут, обычно по телефону или Zoom без видео.

На что HR смотрит, кроме ответов?

На вменяемость, на готовность учиться, на стабильность (не прыгаешь ли каждые полгода между работами / курсами). Для Junior из вуза или после курсов важна мотивация: почему именно разработка, почему Java, что делал руками. Чёткие короткие ответы лучше длинных монологов. Если HR спрашивает «есть ли вопросы», задавай — спроси про процесс адаптации, про менторство, про технологии команды.

ВНИМАНИЕ • Готовность к офису — фильтр

Вакансия офисная, удалёнки нет даже частичной. Если ты живёшь не в СПб или не готов ездить на Пионерскую каждый день — этот вопрос HR закроет разговор сразу. Заранее проверь, сколько ехать от твоего дома до Коломяжского, 33к2. На собеседовании уверенно скажи: «да, готов, рассматриваю офис как плюс — общение с командой важно для Junior».

► Этап 2 — техническое интервью

Кто проводит техническое интервью?

Чаще всего тимлид направления + либо СТО, либо senior разработчик команды. Иногда без СТО, только тимлид. Атмосфера обычно спокойная — компания не банковская, без давления и

стрессовых техник. Спрашивают по конкретным темам по очереди: Java Core, коллекции, JVM, многопоточность, алгоритмы, БД. В конце — практическая задача на коде.

Какие блоки точно будут на техническом?

Java Core (примитивы, Object, equals/hashCode, String, Optional) — всегда. ООП с примерами и SOLID — всегда. Коллекции (HashMap внутри, ArrayList vs LinkedList) — почти всегда. JVM минимально (heap / stack / GC) — часто. Алгоритмы и обход деревьев — точно, это в требованиях вакансии. SQL базовый — точно, ERP без БД не работает. Тесты (JUnit / Mockito) — спросят, особенно про юнит-тесты.

Будет ли тестовое задание?

Может быть, не обязательно. Для Junior часто заменяют живой задачей на собеседе — пишешь код в IDE на компьютере собеседующего или в общем документе. Если тестовое всё-таки дадут, оно будет небольшое: реализовать структуру данных (свой ArrayList, стек, очередь), написать простой REST-сервис, парсер CSV или JSON. Срок — 1–3 дня.

Спрашивают ли что-то нестандартное?

Иногда — логические задачи или вопросы на сообразительность («как определить фальшивую монету за 2 взвешивания», «сколько мячей помещается в школьный автобус»). Это не часто, но возможно. Главное — не зависать молча, рассуждать вслух. На IT-собесах ценят процесс мышления, а не правильный ответ с первой попытки.

Как себя вести во время технического?

Если не знаешь ответ — не выдумывай, скажи честно «не сталкивался, но логично предположить, что...». Если что-то знаешь поверхностно — обозначь это: «слышал термин, но в практике не использовал». Хорошие Junior-кандидаты честны: senior-собеседующие сразу видят, когда ты пытаешься натянуть знания, и это минус. Лучше показать знание границ своего опыта, чем городить ложь.

► Этап 3 — тестовое (если будет)

Что обычно дают в тестовом для Junior?

Три типичных варианта: (1) реализовать структуру данных с нуля — свой ArrayList или LinkedList с методами add, get, remove. (2) Написать маленький REST-сервис на Java SE с одним эндпоинтом, который что-то парсит и возвращает JSON. (3) Парсер — на вход CSV или строка «key1=value1;key2=value2», на выходе структура и операции с ней. Срок 1–3 дня, в реальности занимает 4–6 часов работы.

На что смотрят при проверке тестового?

Чистота кода (читаемые имена переменных, отсутствие магических чисел, разбиение на методы), наличие тестов (хотя бы 3–5 юнитов через JUnit 5), корректность работы на edge cases (пустой вход, null, отрицательные числа), README с описанием решения. Бонус — продуманная архитектура: разделение на слои, интерфейсы для абстракций, обработка исключений. Не дописывай Spring Boot, если в задании про чистую Java SE.

Совет по структуре тестового: сделай отдельный пакет model для классов с данными, service для бизнес-логики, util для парсеров. Покрой 3–5 юнит-тестами через JUnit 5. В README напиши, что реализовал, какая сложность алгоритма, какие edge cases учёл, как запустить. Это уровень Middle+ для тестового — впечатлишь команду.

03 Стек по вакансии: что готовить

Стек вакансии Junior — намеренно базовый. Конкретный фреймворк (Spring, Hibernate) в требованиях не назван — у компании свой framework, его будут учить тебя на месте. На собеседовании проверяют фундамент Java, без которого ни один фреймворк не освоить.

► Что обязательно к подготовке

Тема	Уровень подготовки	Где спросят
Java SE	Уверенно — без подсказок	Тех. собес, тестовое
ООП и принципы	С примерами из жизни / кода	Тех. собес
SOLID, DRY, KISS	Расшифровка + пример	Тех. собес
Коллекции	HashMap внутри, ArrayList vs LinkedList	Тех. собес
JVM / память / GC	На пальцах	Тех. собес
Многопоточность	Базово (Thread, synchronized, volatile)	Тех. собес
Алгоритмы и Big-O	Сложности коллекций + классика	Тех. собес, live coding
Обход деревьев	In/Pre/Post/Level-order — код наизусть	Live coding
SQL базовый	SELECT, JOIN, GROUP BY, агрегаты	Тех. собес
JDBC	Statement vs PreparedStatement	Тех. собес
Тесты (JUnit + Mockito)	Базово	Тех. собес
Git	Базовые команды	HR + тех. собес
Maven	pom.xml, фазы жизненного цикла	Тех. собес

► Что желательно (плюс к offer)

Pet-проекты на Java — даже учебные: backend на чистой Java SE, REST через HttpServer, парсер CSV или JSON, мини-чат через сокет. Главное — твой код, который ты сам объяснишь.

Fullstack web-разработка базово — HTML, CSS, JavaScript, представление о DOM, fetch, JSON. Компания делает веб-клиент, понимание фронта — плюс.

PostgreSQL — основной СУБД продукта. Базовый SQL обязателен, но если знаешь индексы и EXPLAIN — отличный сигнал.

Понимание ERP-домена — что это, какие модули, чем отличается от банка и e-commerce. Это покажет осознанный выбор компании.

Знакомство с Selenium / Selenide / Playwright — потому что в обязанностях упомянуты «визуальные тесты». Не обязательно уметь, но понимать слова «UI-тест», «headless-браузер».

► Чего НЕ ждут от Junior

Знания Spring Boot 3, Hibernate, JPA на глубоком уровне — у компании свой framework, спросят базу Java вместо этого.

Опыта коммерческой разработки — допустимы только pet-проекты.

Опыта с Kafka, RabbitMQ, Kubernetes — это уровень Middle.

Архитектурных решений и system design — для Junior это не стандарт.

Знания конкретных СУБД-нюансов (MVCC, EXPLAIN ANALYZE, плановщик запросов) — спросят максимум базовое.

ВНИМАНИЕ • Особенности этой вакансии

60 процентов времени — на Java Core, коллекции, ООП, SOLID. 20 процентов — на алгоритмы и обход деревьев. 10 процентов — на SQL базовый. 10 процентов — на остальное (JVM, многопоточность, паттерны, тесты, git). Если за вечер успеешь пройти первые два блока — закроешь 80 процентов вопросов на собеседе.

04 Java Core

Это база, без которой ни один Junior-собес не пройти. В Бизнес Технологиях нет Spring Boot — они смотрят, насколько ты владеешь самим языком. Все вопросы ниже типичны для российских собесов уровня Junior; знание ответов даёт стабильную базу для перехода в любую Java-команду.

► Примитивы и обёртки

Сколько примитивных типов в Java и какие?

Восемь: byte (1 байт), short (2), int (4), long (8) — это целые; float (4) и double (8) — вещественные; char (2 байта, символ Unicode) и boolean. Каждый примитив имеет тип-обёртку: Integer, Long, Boolean и так далее. Примитивы хранятся в стеке (если это локальная переменная) и не могут быть null. Обёртки — это объекты в heap, ссылки на них могут быть null.

Чем опасны типы-обёртки?

Тремя вещами. Первое — NullPointerException при автораспаковке: если Integer x = null, и ты пишешь int y = x, поймашь NPE. Второе — расход памяти: Integer занимает 16 байт против 4 байт у int, для больших коллекций это ощутимо. Третье — кэш Integer от -128 до 127: для значений в этом диапазоне Integer.valueOf вернёт один и тот же объект, и == даст true; для значений вне диапазона — разные объекты, и == даст false. Это классическая ловушка собеседа.

Где живут примитивы и где живут объекты?

Примитивы как локальные переменные методов лежат в стеке вместе с другими данными вызова — это быстро и автоматически освобождается при возврате из метода. Если примитив является полем класса, он лежит в heap внутри объекта-владельца. Объекты всегда в heap, а в стеке только ссылки на них. Статические поля (включая примитивы) живут в Metaspace, в области класса.

► Object, equals и hashCode

Какие методы есть у Object?

Их девять: equals, hashCode, toString, getClass, clone, finalize (deprecated с Java 9), wait (три перерывки), notify, notifyAll. Из них на собеседе чаще всего спрашивают про первые три — equals, hashCode, toString. Остальные про многопоточность и про reflection.

Что возвращает toString по умолчанию?

Строку вида ИмяКласса@хэшкод_в_шестнадцатеричном — например, com.example.User@1b6d3586. Это не равно hashCode объекта напрямую: метод Object.toString вызывает Integer.toHexString(System.identityHashCode(this)), то есть идентичность объекта, а не результат твоего переопределённого hashCode. Если переопределяешь equals/hashCode, обязательно переопределяй и toString — это сильно облегчает отладку и логи.

Какой контракт у equals и hashCode?

Три правила. Первое — если a.equals(b) равно true, то a.hashCode должен быть равен b.hashCode (обратное не обязательно). Второе — equals должен быть рефлексивным (a.equals(a) = true), симметричным (a.equals(b) ⇔ b.equals(a)), транзитивным (a=b, b=c ⇒ a=c) и согласованным (повторный вызов возвращает тот же результат). Третье — a.equals(null) должно вернуть false, а не бросать NPE. Нарушение этих правил ломает HashMap, HashSet и любые коллекции на хешах.

Как правильно писать equals?

По шаблону: (1) если `this == other`, вернуть `true` (быстрая проверка идентичности). (2) Если `other == null`, вернуть `false`. (3) Если `getClass() != other.getClass()` или `other instanceof` не подходит — вернуть `false`. (4) Кастуй `other` к своему типу. (5) Сравни все значимые поля через `Objects.equals` (для объектов) или `==/Double.compare` (для примитивов). Параллельно переопредели `hashCode` через `Objects.hash(field1, field2, ...)` — это стандартный путь.

Когда хэш-коды равны, а equals не равны — как такое возможно?

Hash-коды — это `int`, то есть всего 4 миллиарда возможных значений на бесконечное число возможных объектов. По принципу Дирихле обязательно найдутся два разных объекта с одинаковым `hashCode` — это называется коллизия. `HashMap` справляется с этим: при коллизии в одном бакете лежат несколько элементов в виде связанного списка или дерева, и при `get` вызывается `equals` для точного сравнения. Поэтому равные `hashCode` — это норма, неравные `equals` при равных `hashCode` — тоже норма.

► String

Почему String immutable?

Это сделано из соображений безопасности, потокобезопасности и оптимизации. `Immutable String` может безопасно разделяться между потоками без синхронизации, может быть ключом в `HashMap` (`hashCode` не меняется), может попадать в `String Pool` и переиспользоваться. Если бы `String` была изменяемой, передача пути к файлу или пароля в метод позволяла бы изменить значение под капотом — это дыра в безопасности.

Что такое String Pool?

Это специальная область внутри `heap`, где JVM хранит уникальные `String`-литералы. Когда ты пишешь `String s = "hello"`, строка попадает в `pool`, и при повторном литерале `"hello"` в любом месте программы возвращается ссылка на тот же объект. Это экономит память. `String s = new String("hello")` создаёт отдельный объект в `heap` мимо `pool`, а метод `s.intern()` возвращает каноническую версию из `pool`.

String vs StringBuilder vs StringBuffer — что выбрать?

`String` — `immutable`, конкатенация в цикле через `+` создаёт `N+1` промежуточных объектов и медленная. `StringBuilder` — изменяемый, не потокобезопасен, используй для конкатенации в одном потоке (стандартный случай). `StringBuffer` — то же самое, но методы `synchronized`; нужен только если действительно несколько потоков пишут в одну строку, что встречается редко. По умолчанию выбирай `StringBuilder`.

Почему сравнение строк через == опасно?

Потому что `==` сравнивает ссылки, а не содержимое. `String a = "hello"; String b = "hello"; a == b` даст `true` (оба из `pool`). Но `String c = new String("hello"); a == c` даст `false` — это другой объект в `heap`. Правильное сравнение всегда через `equals`. Это классическая ловушка на собеседах и баг в коде Junior-разработчиков.

► Optional

Зачем нужен Optional?

Чтобы явно показать в сигнатуре метода, что результат может отсутствовать. `Optional<User> findUser` сразу говорит читающему: «может вернуть пусто, обработай это». Это лучше, чем возвращать `null`, потому что заставляет вызывающего явно проверить через `ifPresent / orElse / map`, а не забыть и поймать `NPE`. `Optional` существует с Java 8 и предназначен прежде всего для возвращаемых значений, а не для полей класса.

orElse vs orElseGet — в чём разница?

`orElse(defaultValue)` вычисляет `defaultValue` всегда, даже если `Optional` не пуст — это `eager`. `orElseGet(Supplier)` вызывает `supplier` только если `Optional` пуст — это `lazy`. Если `default` — это дорогая операция (запрос к БД, новый объект), используй `orElseGet`, иначе он будет вызываться зря. На собеседовании любят этот вопрос — показывает, что ты не просто заучил API, а понимаешь `execution`.

► Final, finally, finalize

Чем отличаются final, finally и finalize?

Три разных слова, не связаны функционально. `final` — модификатор для класса (нельзя наследовать), метода (нельзя переопределить) или переменной (нельзя переопределить после инициализации). `finally` — блок в `try-catch-finally`, который выполняется всегда независимо от исключения, используется для освобождения ресурсов. `finalize` — метод `Object`, который вызывается GC перед удалением объекта (`deprecated` с Java 9, не использовать; ему на замену `try-with-resources` и `AutoCloseable`).

Что такое effectively final?

Переменная, которая не объявлена как `final`, но фактически не изменяется после инициализации — компилятор считает её `effectively final`. Это важно для лямбд и анонимных классов: они могут захватывать только `final` или `effectively final` переменные. Если в лямбде ты пытаешься изменить захваченную переменную, компилятор не даст. Это сделано, чтобы избежать гонок данных в параллельном коде.

► Модификаторы доступа

Какие модификаторы доступа есть в Java?

Четыре уровня в порядке расширения: `private` (только внутри класса), `default` или `package-private` (без модификатора — виден внутри пакета), `protected` (виден в пакете и наследникам в других пакетах), `public` (виден отовсюду). `Default` — это отсутствие модификатора, не ключевое слово. Самое узкое — `private`; этим пользуйся по умолчанию и расширяй только если действительно нужно.

Как работает protected через пакеты?

`Protected`-член доступен из любого класса того же пакета (как `default`) плюс из подклассов в других пакетах, но только через ссылку на свой тип. Тонкость: если ты подкласс в другом пакете, ты можешь обратиться к `protected`-полю только через ссылку типа подкласса (или его потомка), а не через ссылку на родителя. Это путаница, на собеседовании редко спрашивают глубоко, но факт стоит знать.

► Static и final

Что такое static-поле?

Поле, принадлежащее не объекту, а классу. Существует одно значение для всех экземпляров класса, инициализируется один раз при загрузке класса, живёт в `Metaspace`. Доступ — через имя класса: `MyClass.field`. `Static`-поля часто используют для констант (вместе с `final`), кэшей, счётчиков. Опасность — глобальное состояние, которое сложно тестировать и которое не потокобезопасно по умолчанию.

Что значит final для класса, метода и переменной?

Для класса `final` значит, что от него нельзя наследоваться — пример `String`, `Integer`. Для метода `final` значит, что подкласс не может его переопределить — используется, когда логика метода

критична и должна оставаться неизменной. Для переменной `final` значит, что после первого присваивания её нельзя менять — для примитива неизменно значение, для ссылки неизменна ссылка (но содержимое объекта может меняться, если объект изменяемый).

► Абстрактный класс и интерфейс

Чем отличаются абстрактный класс и интерфейс?

Абстрактный класс может иметь поля состояния, конструкторы, реализованные методы и наследуется один (`single inheritance`). Интерфейс описывает поведение, исторически не имеет состояния (поля только `public static final` = константы), реализаций может быть много через `implements`. С Java 8 в интерфейсах появились `default`-методы и `static`-методы; с Java 9 — `private`. На практике интерфейс — это контракт, абстрактный класс — это шаблон с общей логикой для группы родственных классов.

Что такое `default`-метод в интерфейсе?

Метод интерфейса с реализацией, помеченный ключевым словом `default`. Появился в Java 8, чтобы можно было добавлять новые методы в существующие интерфейсы (например, в `Collection` — `stream()`, `forEach()`) без необходимости править все классы-реализации. Класс-реализатор может переопределить `default`-метод, если хочет. Опасность — `diamond problem`: если класс реализует два интерфейса с одноимёнными `default`-методами, компилятор требует явного выбора через `InterfaceName.super.method()`.

Что такое поля в интерфейсе?

Все поля интерфейса по умолчанию `public static final` — то есть это константы, общие для всех. Их нельзя сделать `private` или изменяемыми. Так интерфейс остаётся контрактом, а не носителем состояния. Если очень нужно общее состояние — это значит, что тебе нужен абстрактный класс, а не интерфейс.

► Generics

Что такое `generics` и `type erasure`?

`Generics` — это параметризованные типы в стиле `List<String>`, появились в Java 5 для типобезопасности коллекций без приведения типов. `Type erasure` означает, что во время компиляции дженерик-информация удаляется: на уровне байткода `List<String>` и `List<Integer>` — это одинаковый класс `List<Object>`. Поэтому нельзя сделать `new T()`, нельзя `instanceof T`, нельзя массив `T[]`. Это компромисс ради обратной совместимости с `pre-Java-5` кодом.

Можно ли в `generics` использовать примитивы?

Нет. `List<int>` запрещён, разрешён только `List<Integer>`. Причина — `type erasure`: внутри `generics` всё становится `Object`, а `Object` не может содержать примитив. Это та же причина, почему обёртки нужны: чтобы примитивы могли участвовать в дженериках, коллекциях, `Optional`, `Stream`. Для производительности в Java 16+ есть `proposal Project Valhalla` с `value types`, который должен это исправить.

► Лямбды и функциональные интерфейсы

Что такое функциональный интерфейс?

Интерфейс с ровно одним абстрактным методом. Может иметь сколько угодно `default`- и `static`-методов — это не считается. Помечается аннотацией `@FunctionalInterface` для явности и проверки компилятором. Примеры из стандартной библиотеки: `Runnable` (`run`), `Callable` (`call`), `Function` (`apply`), `Predicate` (`test`), `Consumer` (`accept`), `Supplier` (`get`). Лямбда — это сахар для реализации функционального интерфейса одной строкой.

Какое свойство у локальной переменной, захваченной лямбдой?

Она должна быть `final` или `effectively final` — то есть фактически не изменяться после инициализации. Если попробуешь изменить захваченную переменную внутри или снаружи лямбды, компилятор не даст. Это связано с тем, как лямбда реализуется под капотом: захваченные переменные копируются в скрытое поле объекта-лямбды, и одновременное изменение в двух местах создало бы гонку данных.

► Stream API

Что такое Stream API и зачем?

`Stream` — это абстракция над последовательностью элементов с операциями `filter`, `map`, `reduce` в декларативном стиле. Появился в Java 8. Главная фишка — `lazy evaluation`: промежуточные операции (`filter`, `map`, `flatMap`, `sorted`) ничего не вычисляют до момента терминальной операции (`collect`, `count`, `forEach`, `reduce`). Это позволяет JVM оптимизировать пайплайн в один проход. `Stream` одноразовый — нельзя повторно использовать после терминальной операции.

Чем `map` отличается от `flatMap`?

`map` преобразует каждый элемент в другой элемент один-к-одному: `Stream<String> → Stream<Integer>` через `String::length`. `flatMap` преобразует каждый элемент в стрим и склеивает все стримы в один — это один-ко-многим. Типичный пример: `Stream<List<Integer>>` через `flatMap(List::stream)` превращается в `Stream<Integer>`. Если у тебя вложенная коллекция и нужно «выровнять» — это `flatMap`.

► Чтение файлов

Как читать файлы в Java?

Три уровня. Самый низкий — байтовые потоки: `FileInputStream` / `FileOutputStream` — для бинарных файлов (картинки, видео). Средний — символьные потоки: `FileReader` / `FileWriter` — для текстовых файлов. С обёрткой буферизации: `BufferedReader` / `BufferedWriter` — для эффективного чтения построчно (`readLine`). Современный путь — NIO: `Files.readAllLines(path)`, `Files.lines(path).stream()` — лаконично, использует современный API.

Что делает `System.exit` и почему не стоит его вызывать?

`System.exit(code)` завершает работу JVM немедленно — все потоки останавливаются, `finally`-блоки могут не успеть выполниться, ресурсы (файлы, соединения) остаются незакрытыми. Это экстренная остановка, не штатное завершение. В production-коде `System.exit` допустим только в `main` и только в коротких CLI-утилитах. Для нормального выхода из приложения используй контролируемое завершение: возврат из `main`, остановка `ExecutorService`, закрытие ресурсов.

► Records и var

Что такое records в Java?

`Records` — это компактный синтаксис для `immutable` классов-данных, появившийся в Java 14 (preview) и `stable` в Java 16. Объявление `record Point(int x, int y) {}` автоматически генерирует приватные `final` поля `x` и `y`, публичный конструктор, методы `x()` и `y()` (геттеры без `get`), `equals`, `hashCode`, `toString`. Использовать для DTO, value-объектов, ключей в коллекциях — везде, где нужны `immutable` бандлы данных без бойлерплейта.

Что такое var?

Ключевое слово для вывода типа локальной переменной, появилось в Java 10. `var list = new ArrayList<String>()` компилируется как `ArrayList<String>`. `var` нельзя использовать для полей класса, аргументов методов, возвращаемых типов — только для локальных переменных.

Удобно при длинных дженериках типа `Map<String, List<Map<Integer, String>>>`, но не стоит злоупотреблять там, где тип неочевиден из контекста — это вредит читаемости.

ВНИМАНИЕ • Частая ловушка с `Integer`

`Integer a = 127; Integer b = 127; a == b` возвращает `true`. `Integer a = 128; Integer b = 128; a == b` возвращает `false`. Причина — `Integer cache`: значения от -128 до 127 кэшируются, остальные создаются заново. Это любимый вопрос на собеседах. Правильное сравнение `Integer` всегда через `equals`.

05 Коллекции

Коллекции — это второй после Java Core блок, который точно будет на собеседовании. В Бизнес Технологиях коллекции используются интенсивно: ERP-система постоянно работает со списками документов, сущностями, контекстами. Junior должен знать иерархию, устройство HashMap, типичные операции и их сложности.

► Иерархия и основные интерфейсы

Какая иерархия коллекций?

На вершине — интерфейс Iterable, от него наследуется Collection. От Collection идут три основные ветви: List (упорядоченная последовательность с индексом — ArrayList, LinkedList, Vector), Set (множество без дубликатов — HashSet, LinkedHashSet, TreeSet), Queue/Deque (очередь и двусторонняя очередь — ArrayDeque, PriorityQueue, LinkedList). Map стоит особняком и не наследуется от Collection — это пары ключ-значение, реализации HashMap, LinkedHashMap, TreeMap, Hashtable.

Почему Map не наследуется от Collection?

Потому что Collection — это набор элементов, а Map — это набор пар ключ-значение. Семантика разная: Collection.add(e) добавляет один элемент, Map.put(k, v) добавляет два связанных значения. У них разные базовые операции, разный жизненный цикл. У Map есть три вспомогательных view-коллекции: keySet, values, entrySet — через них можно работать с Map как с Collection.

► List

ArrayList vs LinkedList — что выбрать?

Почти всегда ArrayList. Он на порядок быстрее на практике, иногда в 10–50 раз: ArrayList хранит элементы подряд в массиве в памяти (cache locality), процессор эффективно подгружает блок в кэш. LinkedList разбрасывает узлы по всему heap, и итерация постоянно промахивается мимо CPU-кэша. LinkedList уместен только если у тебя есть прямая ссылка на узел и нужны частые вставки и удаления в середине — на практике это редкий кейс.

Как устроен ArrayList внутри?

Внутри обычный массив Object[] elementData плюс счётчик size (фактический размер). Начальная capacity — 10 (если не указать иначе в конструкторе). При add: проверяется, есть ли место (size < capacity), если нет — создаётся новый массив capacity * 1.5 и старый копируется через System.arraycopy. Доступ get(i) — O(1) прямой индекс. Удаление в середине — O(n) из-за сдвига элементов. Это объясняет, почему ArrayList быстрый при чтении и медленный при удалении из начала.

Что происходит при переполнении ArrayList?

Внутри вызывается метод grow: создаётся новый массив длиной old * 1.5, через System.arraycopy туда копируется содержимое старого массива, ссылка на elementData переключается на новый. Это означает, что одна операция add иногда становится O(n) вместо амортизированных O(1). Если ты заранее знаешь размер — передай его в конструктор new ArrayList<>(1000), чтобы избежать многократных resize.

► Set и Queue

Как реализован HashSet?

HashSet — это обёртка над HashMap, где ключ — это элемент, а значение — фиктивный плейсхолдер (статический Object PRESENT). Все операции add, remove, contains просто делегируют в HashMap. Поэтому сложность операций — $O(1)$ в среднем, требования к hashCode/equals те же, что у HashMap.

HashSet vs LinkedHashSet vs TreeSet — отличия?

HashSet — без порядка, $O(1)$ операции, реализован через HashMap. LinkedHashSet — сохраняет порядок добавления, $O(1)$ операции, реализован через LinkedHashMap (двусвязный список поверх хеш-таблицы). TreeSet — отсортирован по натуральному порядку или Comparator, $O(\log n)$ операции, реализован через TreeMap на красно-чёрном дереве. Выбирай по требованию к порядку и сложности.

Когда использовать Queue?

Когда нужна FIFO — первый пришёл, первый вышел. Типичные применения: очередь задач в планировщике (Producer-Consumer), BFS-обход графа, буфер сообщений. Стандартные реализации — ArrayDeque (быстрый, на массиве с кольцевым буфером), LinkedList (на связном списке), PriorityQueue (приоритетная очередь на куче — heap). Для большинства случаев бери ArrayDeque.

Что такое Deque?

Двусторонняя очередь — Double Ended Queue. Поддерживает добавление и удаление с обоих концов: addFirst, addLast, removeFirst, removeLast. Может работать как стек (LIFO): addFirst — push, removeFirst — pop. Стандартная реализация ArrayDeque. С Java 6+ ArrayDeque считается предпочтительной заменой устаревшего Stack: и быстрее, и API чище.

► Map

Как устроен HashMap внутри?

Внутри массив бакетов (Node[] table), по умолчанию начальный размер 16. При put: считается hashCode ключа, потом по hashCode выбирается индекс бакета (через побитовое И с длиной массива минус 1). Если бакет пуст — кладём узел. Если занят — добавляем в связный список. При коллизиях восьми и более узлов в одном бакете И размер массива не менее 64 — список превращается в красно-чёрное дерево для ускорения поиска с $O(n)$ до $O(\log n)$. Load factor по умолчанию 0.75: при заполнении 75 процентов происходит resize массива вдвое.

Как работает put и get у HashMap?

Для put: считаем hashCode ключа → выбираем бакет → если бакет пуст, кладём узел → если занят, идём по списку или дереву и через equals ищем существующий ключ → если есть, заменяем value → если нет, добавляем новый узел. Для get: считаем hashCode ключа → находим бакет → идём по списку или дереву и через equals находим ключ → возвращаем value. В среднем оба за $O(1)$, в худшем (плохой hashCode и без treeify) — $O(n)$.

Можно ли null-ключ в HashMap?

В HashMap — да, ровно один null-ключ; он попадает в нулевой бакет, его hashCode принимается за 0. Значений null может быть сколько угодно. В TreeMap null-ключ запрещён — попытка put(null, v) бросит NullPointerException, потому что TreeMap для сортировки вызывает compareTo на ключе, а у null его нет. В Hashtable (устаревший, не использовать) null запрещён ни как ключ, ни как значение.

Что будет, если использовать мутабельный ключ в HashMap?

Беда. При put hashCode ключа считается в момент вставки и определяет бакет. Если ты потом изменяешь поля ключа, его hashCode становится другим, но узел остаётся в старом бакете. При

get с тем же изменённым ключом ты пойдёшь в новый бакет (по новому hashCode) и не найдёшь свой узел — элемент потерян, хотя он физически в Map. Поэтому ключи должны быть immutable: String, Integer, ваш собственный класс с final-полями.

► Сравнения и итерации

Comparable vs Comparator?

Comparable — это интерфейс, который реализует сам класс, чтобы определить «натуральный» порядок: метод compareTo(other), используется в TreeSet, TreeMap, Collections.sort. У String, Integer, Date, LocalDate уже реализован. Comparator — это внешний объект для сортировки по любой логике без изменения класса: можно сортировать тот же User по возрасту в одном месте и по имени в другом, передавая разные Comparator в sort. С Java 8 Comparator стал функциональным интерфейсом — можно писать через лямбды: Comparator.comparing(User::getAge).

Что такое ConcurrentModificationException?

Исключение, которое бросает Iterator, если коллекцию изменили во время итерации не через сам итератор. Внутри ArrayList и HashMap есть счётчик modCount, увеличивающийся при каждом add/remove. Итератор запоминает modCount при создании; при next проверяет — если изменилось, бросает CME. Это fail-fast поведение: лучше упасть быстро, чем работать с неконсистентным состоянием. Обойти можно через iterator.remove() (он сам обновляет счётчик) или через CopyOnWriteArrayList в многопоточном коде.

Iterator vs ListIterator — отличия?

Iterator — базовый интерфейс с методами hasNext, next, remove. Идёт только вперёд по любой Collection. ListIterator расширяет Iterator для List: добавляет hasPrevious, previous (можно идти назад), add, set (можно вставлять и заменять во время итерации), nextIndex, previousIndex (знаешь текущий индекс). На практике ListIterator используется редко — современный код предпочитает forEach и Stream.

ЛОВУШКА • Mutable ключ в HashMap

Если положить в HashMap ключ-объект и потом изменить его поле, влияющее на hashCode (например, добавить элемент в Set, который используется как ключ), элемент станет невидим — он будет лежать в Map, но get не сможет его найти. Это классическая Junior-ловушка. Правило: ключи всегда immutable.

06 ООП, SOLID и принципы

ООП и SOLID — это блок, который в Бизнес Технологиях точно спросят. Junior-собес обязательно идёт через «расскажи 4 принципа ООП», «расскажи SOLID», «приведи пример из своего пет-проекта». Заученные определения не катят: на каждый принцип нужно иметь маленький пример.

► Четыре принципа ООП

Какие 4 принципа ООП?

Инкапсуляция, наследование, полиморфизм, абстракция. Это база, которая считается обязательной для любого Java-разработчика. На собеседе ожидают, что ты не просто назовёшь их, а коротко объяснишь каждый своими словами и приведёшь маленький пример из жизни или из своего пет-проекта.

Что такое инкапсуляция?

Скрытие внутреннего состояния объекта за публичным интерфейсом. Поля делаешь `private`, доступ — через геттеры и сеттеры (или вообще без сеттеров, если хочешь `immutable`). Цель — контроль над тем, как изменяется состояние: можно добавить валидацию в сеттере, логирование, расчётные поля. Внешний код работает с объектом через API и не зависит от деталей реализации.

Что такое наследование?

Механизм, при котором один класс получает поля и методы другого. В Java наследование одиночное: класс наследуется от ровно одного родителя через `extends`. Цели — переиспользование общего поведения и построение иерархии типов. Опасность — крепкая связь: изменение родителя ломает наследников. Современный совет — «*favor composition over inheritance*»: предпочитай композицию (передавать объекты внутрь как поля) наследованию.

Что такое полиморфизм?

Возможность работать с разными классами через единый интерфейс или общего родителя. Если у тебя интерфейс `Shape` с методом `area`, ты можешь иметь `Circle`, `Square`, `Triangle` — и в коде писать `Shape shape = new Circle(); shape.area()`. Метод вызывается по реальному типу объекта (виртуальный вызов), даже если ссылка объявлена базовым типом. Это позволяет писать обобщённый код, который работает с любым новым подтипом без изменений.

Что такое абстракция?

Выделение существенных свойств и игнорирование деталей. В коде это выражается через `abstract class` и `interface`: ты описываешь «что объект умеет», не вдаваясь в «как именно». На уровне дизайна абстракция — это умение видеть сущности предметной области (`User`, `Order`, `Invoice`) и моделировать их связи, отбрасывая лишние детали.

Что такое DRY, KISS, YAGNI?

Три принципа поверх ООП. DRY — *Don't Repeat Yourself*: не дублируй код, выноси повторяющуюся логику в общий метод/класс. KISS — *Keep It Simple, Stupid*: предпочитай простое решение сложному, если оно решает задачу. YAGNI — *You Aren't Gonna Need It*: не пиши «на будущее», добавляй абстракции, только когда они реально нужны. Эти принципы — про здравый смысл в проектировании; на собеседе их полезно знать как сокращения.

► SOLID

Что означают все буквы SOLID?

S — Single Responsibility (единая ответственность): у класса должна быть одна причина изменяться. O — Open/Closed (открыт/закрыт): код открыт для расширения, закрыт для модификации (новое поведение добавляешь через подклассы или композицию, а не правя старый код). L — Liskov Substitution: подкласс должен быть взаимозаменяем с родителем без поломки поведения. I — Interface Segregation: лучше много мелких интерфейсов, чем один толстый. D — Dependency Inversion: зависимости направлены от деталей к абстракциям, а не наоборот.

Что такое Single Responsibility и зачем нужен?

Класс должен решать одну задачу. Например, UserService не должен и валидировать пароль, и отправлять email, и сохранять в БД — лучше выделить EmailService, UserRepository, PasswordValidator. Тогда изменение в логике писем не затронет логику сохранения. Принцип проверяется вопросом: «сколько причин может вынудить меня изменить этот класс?». Если их несколько и они независимы — разделяй.

Что такое Dependency Inversion и зачем нужен?

Высокоуровневый код не должен зависеть от низкоуровневого; оба должны зависеть от абстракции. Простыми словами: вместо `new MysqlUserRepository()` внутри сервиса — принимай UserRepository (интерфейс) в конструкторе извне, а реализацию (Mysql, Postgres, In-Memory) подсовывай через DI-контейнер или передавай руками в тестах. Это позволяет менять реализацию без правки сервиса и легко мокать в юнит-тестах.

Что такое Liskov Substitution и почему его любят спрашивать?

Если код работает с объектом базового типа, он должен работать с любым подтипом без сюрпризов. Классический контрпример: класс Rectangle с `setWidth` и `setHeight`, и подкласс Square, у которого `setWidth` заодно меняет `height`. Код, написанный на Rectangle и ожидающий, что `width` и `height` независимы, сломается на Square. Решение — Square не должен наследоваться от Rectangle, это разные концепции.

Какое отличие между Interface Segregation и Single Responsibility?

Похожи, но про разное. SRP — про размер ответственности одного класса. ISP — про размер интерфейса: лучше дать клиенту узкий интерфейс с теми методами, которые он реально использует, чем толстый интерфейс «на все случаи жизни». Если у класса есть метод, который никогда не вызывается одной группой клиентов — это сигнал разделить интерфейс. Принцип защищает от ситуаций «реализую интерфейс, но половина методов выкидывает `UnsupportedOperationException`».

► Объект и класс

Что такое объект в ООП?

Объект — это экземпляр класса, существующий в памяти, который имеет состояние (поля) и поведение (методы). В Java объекты живут в heap, доступ к ним через ссылки. Класс — это «чертёж» объекта, его описание; объект — это «материализация» этого чертежа. Из одного класса можно создать сколько угодно объектов, каждый со своим состоянием.

Все ли объекты в Java наследуются от Object?

Да. Любой класс, явно или нет, является наследником Object — даже если ты не пишешь `extends`. Object — единственный класс, у которого нет родителя. Поэтому все объекты имеют методы `equals`, `hashCode`, `toString`, `getClass` — они унаследованы от Object и могут быть переопределены. Это провокационный вопрос на собеседовании: отвечать уверенно — «да, все».

На собесе хорошо звучит пример из жизни: «У меня в пет-проекте был сервис заказов, который и валидировал заказы, и отправлял уведомления, и сохранял в БД. После SRP я разделил на `OrderValidator`, `NotificationService`, `OrderRepository` — стало легче тестировать и добавлять новые типы уведомлений». Если есть свой пример из кода — это сильно лучше абстрактного определения.

07 Исключения

Исключения в Java — тема, которую любят на Junior-собесах. Базовая иерархия, checked vs unchecked, try-with-resources — обязательный набор. Часто спрашивают также «можно ли поймать Error» — практически нет, потому что Error это про падение JVM.

► Иерархия исключений

Какая иерархия исключений в Java?

На вершине Throwable. От него ветвятся два класса: Error и Exception. Error — критические ошибки JVM (OutOfMemoryError, StackOverflowError), которые приложение не должно ловить. Exception делится на checked (IOException, SQLException — компилятор требует обработать) и RuntimeException (NullPointerException, IllegalArgumentException — необязательные к обработке). Все наследники RuntimeException автоматически unchecked.

Checked vs unchecked — в чём разница?

Checked — компилятор заставляет либо обернуть в try-catch, либо объявить throws в сигнатуре метода. Это IOException, SQLException, ClassNotFoundException и так далее. Используются для предсказуемых ошибок, которые вызывающий должен обработать. Unchecked (RuntimeException и его наследники) — компилятор не требует ничего; обычно это баги или нарушения контракта (NPE, IllegalArgumentException, ArithmeticException). Современный тренд — больше unchecked-исключений, меньше checked, потому что checked засоряют сигнатуры.

Что такое try-with-resources?

Синтаксис Java 7+ для автоматического закрытия ресурсов. try (var reader = new BufferedReader(new FileReader("file"))) { ... } — после блока reader.close() будет вызван автоматически, даже при исключении. Ресурс должен реализовывать AutoCloseable (Closeable наследуется от AutoCloseable). Это замена руками написанному finally { if (reader != null) try { reader.close(); } catch ... }, который раньше был стандартом и от которого многие ошибки.

Что такое AutoCloseable?

Интерфейс с одним методом close, который позволяет использовать класс в try-with-resources. close может бросать Exception (Closeable бросает только IOException). Реализуй AutoCloseable, если твой класс владеет внешним ресурсом (соединение с БД, файл, сокет, поток), который нужно явно освобождать.

► Конкретные исключения

OutOfMemoryError — это Error или Exception?

Error. Это критическая ситуация — у JVM кончилась куча или другая память, продолжать работу обычно нельзя. Технически OOM можно поймать через catch (Error e), но это плохая практика: после OOM состояние памяти непредсказуемо, поток мог не доделать важную операцию, неконсистентность данных. Лучше дать JVM упасть, разобраться по heap dump, исправить корень проблемы.

IndexOutOfBoundsException — checked или unchecked?

Unchecked — наследник RuntimeException. Бросается при обращении за пределы массива, списка или строки. Это баг программиста, а не предсказуемая ошибка. Компилятор не заставит тебя его ловить — он рассчитывает, что ты пишешь правильный код. Похожие — NullPointerException, ClassCastException, ArithmeticException — тоже unchecked.

Можно ли создать собственное исключение?

Да, любой может `extends RuntimeException` (для `unchecked`) или `extends Exception` (для `checked`). Обычно делают: класс с конструктором, принимающим `String message` и `cause`, передающим их в `super`. Используется для бизнес-исключений: `UserNotFoundException`, `OrderAlreadyPaidException` — что-то осмысленное в твоей предметной области, а не общий `RuntimeException`.

► Best practices

Зачем нужен блок `finally`, если есть `try-with-resources`?

`finally` нужен для логики, которая должна выполняться независимо от исключения и не связана с закрытием ресурсов: разлогирование, метрики, освобождение блокировок (`ReentrantLock` не работает в `try-with-resources`, потому что `lock` не `AutoCloseable`). `Try-with-resources` покрывает 90 процентов кейсов с ресурсами и значительно упрощает код, но `finally` всё ещё нужен в специальных случаях.

Что плохого в `catch (Exception e)`?

Слишком широко — поймаешь и `NPE` (бага), и `IOException` (внешнюю ошибку), и любое будущее исключение, которое ещё не предусмотрел. Это маскирует баги: программа продолжит работать в неконсистентном состоянии вместо того, чтобы упасть и заставить тебя разобраться. Правило — лови самое узкое исключение, которое можешь обработать. `Catch (Exception e)` допустим только на верхних уровнях (в `main`, в обработчике запросов) для логирования и `graceful shutdown`.

08 HTTP и REST

REST и HTTP базово спрашивают на Junior-собесах, особенно если в вакансии упомянуты «модульные и визуальные тесты». В Бизнес Технологиях это нужно для понимания, как фронт-клиент Global ERP общается с сервером приложений. Глубоко не копают, нужны базовые понятия.

► Структура HTTP

Из чего состоит HTTP-запрос?

Из четырёх частей: метод (GET, POST, PUT, PATCH, DELETE), URL (адрес ресурса), заголовки (Content-Type, Accept, Authorization, User-Agent — мета-информация в формате ключ-значение), тело (опционально — данные для POST, PUT, PATCH; в GET тела обычно нет). HTTP-ответ имеет похожую структуру: статус-код (200, 404, 500), заголовки, тело.

Какие есть HTTP-методы?

GET — получить данные, без побочных эффектов. POST — создать или выполнить операцию. PUT — заменить ресурс целиком. PATCH — частично обновить. DELETE — удалить ресурс. HEAD — как GET, но без тела (только заголовки, для проверки существования). OPTIONS — узнать, какие методы поддерживает endpoint. На практике 95 процентов кода — это GET, POST, PUT, DELETE.

Что такое идемпотентность?

Свойство операции, при котором повторный вызов с теми же параметрами даёт тот же результат. Идемпотентные методы: GET (читаешь данные — состояние не меняется), PUT (заменяешь ресурс полностью — повторная замена ничего не меняет), DELETE (удаление уже удалённого = ничего не происходит). Неидемпотентный — POST (каждый POST создаёт новый ресурс или выполняет операцию). Идемпотентность важна для retry-логики: если запрос упал по таймауту, повторять идемпотентные методы безопасно, POST — нет.

► PUT vs PATCH

Чем PUT отличается от PATCH?

PUT заменяет ресурс целиком: ты отправляешь полный объект, сервер записывает его поверх старого; поля, которых нет в запросе, удаляются. PATCH частично обновляет: ты отправляешь только изменяемые поля, остальные остаются как есть. На практике большинство API используют PUT для полной замены и PATCH для частичных правок (например, изменить только email пользователя).

► Статус-коды

Какие есть основные группы статус-кодов?

2xx — успех (200 OK, 201 Created, 204 No Content). 3xx — редиректы (301 Moved Permanently, 304 Not Modified). 4xx — ошибка клиента (400 Bad Request — невалидный запрос, 401 Unauthorized — не авторизован, 403 Forbidden — нет прав, 404 Not Found, 409 Conflict, 422 Unprocessable). 5xx — ошибка сервера (500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout). Junior должен уверенно отличать 401 (не знаем, кто ты) от 403 (знаем, но не пустим).

Чем отличается 401 от 403?

401 Unauthorized — клиент не предоставил валидные учётные данные или они истекли. Сервер не знает, кто это, и предлагает аутентифицироваться. 403 Forbidden — клиент аутентифицирован, сервер знает, кто это, но у пользователя нет прав на эту операцию. На уровне ERP: 401 — токен истёк или его нет; 403 — у обычного бухгалтера нет права смотреть зарплаты директоров.

► REST

Что такое REST?

Representational State Transfer — архитектурный стиль для веб-сервисов, описанный Роем Филдингом в 2000 году. Главные принципы: stateless (каждый запрос автономен, сервер не хранит сессию между ними), client-server (чёткое разделение), cacheable (ответы могут кэшироваться), layered (между клиентом и сервером могут быть прокси), uniform interface (унифицированный интерфейс: ресурсы по URL, операции через HTTP-методы, представление через JSON или XML). REST — это не протокол, а набор соглашений поверх HTTP.

Чем REST отличается от HTTP?

HTTP — это протокол передачи данных. REST — это архитектурный стиль, который использует HTTP как транспорт и его методы как операции. Можно делать веб-сервисы поверх HTTP, не следуя REST (например, отправлять всё через POST с операцией в теле — это SOAP-like). REST — это о том, что URL обозначает ресурс, метод обозначает действие, а коммуникация stateless. То есть HTTP — это «как», REST — это «по каким правилам».

Чем REST отличается от SOAP?

SOAP — это тяжёлый протокол с XML-конвертами, WSDL для описания контракта, поддержкой стандартов безопасности WS-*. Сильная типизация, формальный контракт, сложность. REST — лёгкий стиль на HTTP, обычно JSON, контракт описывается OpenAPI / Swagger, гибкость. SOAP жив в enterprise-системах (банки, страховые, интеграция с госструктурами), REST доминирует во всём остальном современном вебе.

► Заголовки и тело

Какими способами можно передать данные в HTTP-запросе?

Четырьмя. (1) Через URL-путь — /users/42 — для идентификаторов ресурсов. (2) Через query-параметры — /users?page=2&size=10 — для фильтров, пагинации, сортировок. (3) В теле запроса (body) — для POST, PUT, PATCH; обычно JSON. (4) В заголовках — Authorization для токена, Content-Type для типа содержимого, X-Request-Id для трассировки. Чувствительные данные (пароли, токены) — только в заголовках или body, никогда в URL.

Что такое Content-Type?

HTTP-заголовок, указывающий MIME-тип содержимого. Типичные значения: application/json (REST API с JSON), application/xml (XML), application/x-www-form-urlencoded (HTML-форма), multipart/form-data (форма с файлами), text/html (HTML-страница), text/plain (обычный текст). Сервер по Content-Type понимает, как парсить тело, и клиент по Content-Type ответа понимает, как читать данные.

► JSON и XML

JSON vs XML — что выбрать?

Для современного REST API почти всегда JSON: компактнее, быстрее парсится, нативно поддерживается JavaScript в браузерах. XML — для legacy-интеграций, SOAP-сервисов,

документов со сложной структурой (например, SVG, DOCX, RSS). Если строишь новый сервис — JSON. Если интегрируешься с банком или налоговой — скорее всего XML.

CORS (Cross-Origin Resource Sharing) — частый источник проблем при разработке: страница с одного origin (домен + порт + протокол) не может делать запросы на другой origin. Сервер разрешает или запрещает запросы через заголовки Access-Control-Allow-Origin. Junior должен знать, что CORS — это про безопасность браузера, а не серверного кода. На собеседе спросят редко, но если упомянут — отвечай уверенно.

09 Git, Maven и инструменты

Git и Maven — рутинный инструментарий. На собеседовании спрашивают на уровне «знаешь команды», без углубления. Junior должен уверенно делать клон, коммит, пуш, бранч, мерж, рассказывать про pull request и понимать Maven-фазы.

► Git — базовые команды

Какие базовые команды git ты используешь?

git clone — скачать репозиторий с remote. git status — что изменено. git add файл — поставить изменения в index (staging). git commit -m "msg" — закоммитить. git push — отправить на remote. git pull — забрать новое с remote и слить в текущую ветку. git branch — список веток или создать новую. git checkout (или git switch в новых версиях) — переключиться на ветку или коммит. git merge ветка — слить ветку в текущую. git log — история коммитов. Этого хватает на 95 процентов работы.

Чем merge отличается от rebase?

Merge сливает две ветки, создавая merge-коммит (если есть расхождения). История получается ветвистой и сохраняет реальный порядок работы. Rebase «переписывает» твою ветку: берёт твои коммиты, как будто они сделаны поверх последнего коммита target-ветки. История получается линейной и красивой, но это переписывание истории. Правило — не делай rebase публичных веток, в которые контрибютируют другие; rebase только своих локальных веток перед пушем.

Что такое cherry-pick?

Команда git cherry-pick <commit-hash> применяет один конкретный коммит из другой ветки в текущую. Полезно, когда нужен один багфикс из dev в release-ветку, а не вся ветка. Создаёт новый коммит в текущей ветке с теми же изменениями.

Pull Request — что это?

Не команда git, а возможность GitHub/GitLab/Bitbucket: предложение влить твою ветку в основную ветку проекта. Открываешь PR (на GitLab это Merge Request), команда смотрит diff, оставляет комментарии, после approve мержат. Это основной механизм code review в современных командах. Junior должен знать слово, понимать процесс — будут спрашивать про опыт работы в PR.

Git Flow vs Trunk-based?

Git Flow — модель с несколькими долгоживущими ветками: master (production), develop (текущая разработка), feature/*, release/*, hotfix/*. Сложная, контролируемая, подходит для релизов раз в месяц или квартал. Trunk-based — короткие feature-ветки, мерж в main быстро (несколько раз в день), production-релиз с main или ветки release-X. Проще, поддерживает continuous delivery. В Бизнес Технологиях с релизами по графику — скорее всего что-то ближе к Git Flow.

► Maven

Что такое pom.xml?

Project Object Model — главный файл Maven-проекта в формате XML. В нём: groupId, artifactId, version проекта; зависимости (dependencies); плагины (plugins); свойства; build-конфигурация.

Запуск `mvn install` читает `pom.xml`, скачивает указанные зависимости из центрального репозитория, компилирует, тестирует, упаковывает в `jar`.

Какие фазы у Maven?

Стандартный жизненный цикл сборки: `validate` → `compile` → `test` → `package` → `verify` → `install` → `deploy`. `validate` проверяет корректность `pom`. `compile` компилирует `sources`. `test` запускает юнит-тесты. `package` упаковывает в `jar` или `war`. `verify` запускает интеграционные тесты. `install` ставит артефакт в локальный репозиторий `~/m2`. `deploy` выкладывает в `remote` (Nexus, Artifactory). Каждая фаза автоматически запускает все предыдущие.

Какие scopes у зависимостей в Maven?

`compile` — стандартный, виден везде (`compile`, `test`, `runtime`). `provided` — есть на компиляции и тестах, но в `runtime` предоставляется средой (Servlet API в контейнере). `runtime` — нужен только в `runtime`, не для компиляции (драйвер JDBC). `test` — только для тестов (JUnit, Mockito). `system` — устаревший, не использовать. `import` — для импорта BOM-зависимостей (Bill Of Materials).

Что такое транзитивные зависимости?

Если твоя библиотека A зависит от B, а B зависит от C, то C — транзитивная зависимость A для тебя. Maven автоматически подтянет всю цепочку. Проблема — конфликт версий: A хочет C 1.0, другая D хочет C 2.0. Maven выбирает по правилу «ближайшая в графе побеждает» или по порядку объявления. Посмотреть дерево можно через `mvn dependency:tree`. Если конфликт ломает приложение — добавляешь нужную версию явно в `pom`.

Maven vs Gradle?

Maven — XML, фиксированный жизненный цикл, декларативный, проще для Junior. Gradle — на Groovy или Kotlin DSL, гибче, мощнее, инкрементальная компиляция (быстрее на больших проектах), сложнее в начале. Большинство Java-проектов используют Maven, Android — Gradle. Spring Boot можно собирать обоими. На себе достаточно знать оба слова и базовое отличие.

Программа платного канала

Разделы 01–09 выше — это язык и база, полностью открытые. Дальше идёт остальная программа собеса: разделы 10–22. Чтобы ты видел её наглядно, ниже показаны все темы и реальные вопросы по каждому разделу. Сами ответы, разборы и код открываются в Java Jub Pro.

Собес на эту вакансию стоит на трёх китах: Java Core, алгоритмы и обход деревьев. Core ты уже прошёл в открытой части. Алгоритмы (раздел 17), обход деревьев (18) и live coding с решениями (19) — ровно то, на чём чаще всего спотыкаются — в Pro.

Открыто в этом гайде (01–09)	Java Jub Pro (10–22)
Компания, формат собеса, стек	10 · JVM, память и сборщик мусора
Java Core — целиком	11 · Многопоточность
Коллекции	12 · Spring и Spring Boot
ООП, SOLID и принципы	13 · SQL и базы данных
Исключения	14 · JDBC
HTTP и REST	15 · Паттерны проектирования
Git, Maven и инструменты	16 · Тестирование (JUnit, Mockito)
	17 · Алгоритмы и Big-O
	18 · Обход деревьев
	19 · Live coding: 12 задач с решениями
	20 · Code Review с разбором
	21 · ERP-домен и web / fullstack
	22 · Pet-проекты, soft skills, чек-лист

ВНИМАНИЕ · Где именно валят Junior на этом собесе

По разбивке из раздела 03: 60% времени — Java Core, коллекции, ООП (это открытая часть). Но 20% — алгоритмы и обход деревьев, плюс блок live coding, где код пишут при интервьюере. Именно эти 20–30% чаще всего и заваливают — теорию учат, а руками не пишут. Разделы 17–20 закрывают ровно это.

10 JVM, память и сборщик мусора

JAVA JUB PRO · Раздел открывается в платном канале

Темы и вопросы раздела — ниже. Полные ответы, разборы и код — в Java Jub Pro · @Java_Jub_Pro

Базовый блок про устройство JVM — спрашивают «на пальцах», но спрашивают всегда.

► Структура памяти

- Из чего состоит память JVM?
- Где живут примитивы, объекты и статические поля?
- StackOverflowError vs OutOfMemoryError — в чём разница?

► Сборщик мусора (GC)

- Как работает GC в Java?
- Соберутся ли два объекта с циклическими ссылками друг на друга?
- Какие бывают сборщики мусора?
- Что такое сильные, слабые и фантомные ссылки?

► String Pool

- Что такое String Pool и где он живёт?

► Утечки памяти

- Какие бывают утечки памяти в Java?
- Как искать утечки памяти?

11 Многопоточность

JAVA JUB PRO · Раздел открывается в платном канале

Темы и вопросы раздела — ниже. Полные ответы, разборы и код — в Java Jub Pro · @Java_Jub_Pro

Поток, синхронизация и типичные ловушки — этим добивают техническое после Java Core.

▶ Поток и процесс

- Чем поток отличается от процесса?
- Как создать поток в Java?
- Runnable vs Callable — отличия?

▶ **synchronized и volatile**

- Как работает synchronized?
- Что гарантирует volatile?
- Чем volatile отличается от synchronized?

▶ Атомарные операции

- Что под капотом у AtomicInteger?

▶ **wait, notify, deadlock**

- Что такое wait, notify, notifyAll?
- Что такое race condition?
- Что такое deadlock и какие 4 условия?
- Чем deadlock отличается от livelock?
- Что такое ThreadLocal?

12 Spring и Spring Boot

JAVA JUB PRO · Раздел открывается в платном канале

Темы и вопросы раздела — ниже. Полные ответы, разборы и код — в Java Jub Pro · @Java_Jub_Pro

У компании свой framework, но Spring — это плюс к offer и частый вопрос «а как в Spring».

▶ **IoC и DI**

- Что такое IoC?
- Что такое DI и какие способы инъекции?
- Что такое ApplicationContext?

▶ **Создание бинов**

- Чем отличаются @Component, @Service, @Repository, @Controller?
- Как ещё можно создать бин кроме аннотаций?

▶ **Жизненный цикл и scope**

- Какой scope у бинов по умолчанию?
- Как выглядит жизненный цикл бина упрощённо?

▶ **@Transactional**

- Что делает @Transactional?

▶ **Spring Boot**

- Зачем нужен Spring Boot?
- Что делает аннотация @SpringBootApplication?

13 SQL и базы данных

JAVA JUB PRO · Раздел открывается в платном канале

Темы и вопросы раздела — ниже. Полные ответы, разборы и код — в Java Jub Pro · @Java_Jub_Pro

PostgreSQL — основная СУБД продукта, базовый SQL на собесе спросят точно.

► Основы SQL

- Что такое DML, DDL, DCL, TCL?
- Какой порядок выполнения частей SELECT?
- Чем отличается WHERE от HAVING?

► JOIN

- Какие бывают JOIN?
- Когда использовать подзапрос вместо JOIN?

► Агрегаты и GROUP BY

- Какие агрегатные функции есть в SQL?
- Какие правила GROUP BY и SELECT?

► NULL

- Что такое NULL в SQL?

► Ключи и индексы

- В чём разница PRIMARY KEY, UNIQUE и FOREIGN KEY?
- Что такое индекс и зачем?
- Что такое sequence?

► Транзакции

- Что такое транзакция и что такое ACID?
- Какие уровни изоляции?
- Можно ли всегда использовать SERIALIZABLE?

14 JDBC и работа с БД из Java

JAVA JUB PRO · Раздел открывается в платном канале

Темы и вопросы раздела — ниже. Полные ответы, разборы и код — в Java Jub Pro · @Java_Jub_Pro

Как Java ходит в базу напрямую — классический вопрос технического интервью.

▶ Базовые классы JDBC

- Какие основные классы JDBC?
- Statement vs PreparedStatement — отличия?
- Что такое SQL-инъекция?

▶ Жизненный цикл соединения

- Что такое connection pool?

▶ Альтернативы JDBC

- Что такое ORM?
- Hibernate vs JdbcTemplate vs jOOQ — что выбирают?

15 Паттерны проектирования

JAVA JUB PRO · Раздел открывается в платном канале

Темы и вопросы раздела — ниже. Полные ответы, разборы и код — в Java Jub Pro · @Java_Jub_Pro

Три группы паттернов с примерами уровня Junior, плюс DTO и POJO.

▶ Три группы

- На какие группы делятся паттерны?

▶ Порождающие

- Singleton — что это и как реализовать?
- Builder — зачем нужен?
- Factory Method vs Abstract Factory?

▶ Структурные

- Adapter — что это?
- Decorator — что это?
- Proxy vs Decorator — в чём разница?
- Facade — что это?

▶ Поведенческие

- Observer — что это?
- Strategy — что это?

▶ DTO и POJO

- Что такое DTO и POJO?

16 Тестирование: JUnit, Mockito, AssertJ

JAVA JUB PRO · Раздел открывается в платном канале

Темы и вопросы раздела — ниже. Полные ответы, разборы и код — в Java Jub Pro · @Java_Jub_Pro

Как писать тесты и мокать зависимости — и почему в вакансии упомянуты визуальные тесты.

▶ JUnit 5

- Что такое юнит-тест и интеграционный тест?
- Какие основные аннотации в JUnit 5?
- Какие assert-методы есть в JUnit?
- Что такое AssertJ и зачем?
- Чем hard assertions отличаются от soft?

▶ Mockito

- Зачем нужны моки?
- Как работает Mockito?
- Mock vs Spy — отличия?
- Какие основные аннотации в Mockito?

▶ Testcontainers

- Что такое Testcontainers?

▶ Визуальные тесты

- Что такое визуальные тесты (упомянуты в вакансии)?

17 Алгоритмы и Big-O

JAVA JUB PRO · Раздел открывается в платном канале

Темы и вопросы раздела — ниже. Полные ответы, разборы и код — в Java Jub Pro · @Java_Jub_Pro

Сложности коллекций и классические алгоритмы — один из трёх китов этого собеса.

► **Big-O**

- Что такое Big-O нотация?
- Какая сложность операций у ArrayList и LinkedList?
- Какая сложность у HashMap?

► **Структуры данных**

- Стек vs очередь vs дек?
- Что такое связный список и где применяется?
- Что такое двоичное дерево, BST, AVL?
- Что такое hash-таблица и как разрешаются коллизии?

► **Сортировки**

- Какие сортировки знаешь и какая сложность?
- Что такое стабильность сортировки?

► **Классические алгоритмы**

- Как реализовать бинарный поиск?
- Что такое два указателя?
- Что такое DFS и BFS?

► **Жадные алгоритмы и мемоизация**

- Что такое жадный алгоритм?
- Что такое мемоизация?
- Что такое рекурсия и чем опасна?

18 Обход деревьев

JAVA JUB PRO · Раздел открывается в платном канале

Темы и вопросы раздела — ниже. Полные ответы, разборы и код — в Java Jub Pro · @Java_Jub_Pro

Обход деревьев в описании вакансии вынесен отдельной строкой — это главная тема live coding.

▶ Четыре обхода

- Какие бывают обходы дерева?

▶ Класс `TreeNode` и реализация

▶ Сложность и применения

- Какая сложность всех обходов дерева?
- Где применяется in-order?
- Где применяется pre-order?
- Где применяется post-order?
- Где применяется level-order (BFS)?

▶ Итеративная версия in-order

- Как сделать in-order итеративно?

▶ Дополнительные задачи на деревьях

- Как посчитать высоту дерева?
- Как проверить, что дерево сбалансировано?

19 Live coding: Java-задачи

JAVA JUB PRO · Раздел открывается в платном канале

Темы и вопросы раздела — ниже. Полные ответы, разборы и код — в Java Jub Pro · @Java_Jub_Pro

12 разобранных задач уровня собеседа — с готовым кодом и объяснением, что скажет интервьюер.

- 19.1. Палиндром
- 19.2. Реализация ArrayList с нуля
- 19.3. Анаграммы
- 19.4. Подсчёт повторов символов
- 19.5. Поиск дубликатов в массиве
- 19.6. FizzBuzz
- 19.7. Бинарный поиск
- 19.8. Сдвиг массива влево на N позиций
- 19.9. Слияние двух отсортированных массивов
- 19.10. Парсинг строки в Map
- 19.11. Корректность скобок
- 19.12. SQL live-coding

20 Code Review с разбором

JAVA JUB PRO · Раздел открывается в платном канале

Темы и вопросы раздела — ниже. Полные ответы, разборы и код — в Java Jub Pro · @Java_Jub_Pro

Разбор реального кода с типичными ошибками Junior — учит видеть проблемы до ревьюера.

- 20.1. Исходный код
- 20.2. Найденные проблемы
- 20.3. Исправленный вариант

21 ERP-домен и базовый web / fullstack

JAVA JUB PRO · Раздел открывается в платном канале

Темы и вопросы раздела — ниже. Полные ответы, разборы и код — в Java Jub Pro · @Java_Jub_Pro

Что такое ERP, чем домен отличается от банка и e-commerce, плюс база по web — под эту вакансию.

► 21.1. Что такое ERP

- Что такое ERP-система?
- Из каких модулей состоит типичная ERP?
- Что такое импортозамещение в контексте ERP?
- Что такое документооборот в ERP?
- Что такое локализация и кастомизация ERP?
- Что такое длинные бизнес-операции?
- Что такое печатная форма?
- Что такое интеграции в ERP?
- Что такое 152-ФЗ?

► 21.2. Web и fullstack — базовые понятия

- Из чего состоит веб-стек?
- Что такое DOM?
- Что такое AJAX и fetch?
- Что такое JSON и зачем он?
- Что такое SPA?
- Что такое CORS?
- В чём разница cookie и localStorage?
- Что такое аутентификация и авторизация?

22 Pet-проекты, soft skills и финальный чек-лист

JAVA JUB PRO · Раздел открывается в платном канале

Темы и вопросы раздела — ниже. Полные ответы, разборы и код — в Java Jub Pro · @Java_Jub_Pro

Какие пет-проекты собрать под вакансию, как отвечать на soft-вопросы и полный чек-лист готовности.

▶ 22.1. Идеи pet-проектов под ERP-домен

▶ 22.2. Как рассказывать о pet-проекте (метод STAR)

- Что такое метод STAR?
- Что обязательно должно быть в репозитории pet-проекта?

▶ 22.3. Soft-вопросы и как на них отвечать

- Что отвечать на «Расскажи о себе»?
- Почему Java?
- Почему Бизнес Технологии?
- Готов ли к работе в офисе в Санкт-Петербурге?
- Готов ли работать с собственным фреймворком вместо Spring?
- Как ты учишься новому?
- Что отвечать на «Расскажи о самом сложном баге, который чинил»?
- Готов ли работать с legacy-кодом?
- Что нравится в Java и что не нравится?
- Какие у тебя вопросы к нам?

▶ 22.4. Финальный чек-лист подготовки

Это вся программа собеседа.

Разделы 01–09 — открыты. Ответы и код по 10–22 — в платном канале

Java Jub Pro · @Java_Jub_Pro

// continue --> JavaJub Pro

JavaJub